Sri Sivasubramaniya Nadar College of Engineering, Chennai

(An autonomous Institution affiliated to Anna University)

Degree & Branch: M. Tech (Integrated) Computer Science & Engineering
Semester: V
Subject Code & Name: ICS1512 – Machine Learning Algorithms Laboratory
Academic year: 2025–2026 (Odd)
Batch: 2023-2028
Experiment: 2 – Email Spam or Ham Classification using Naïve Bayes, KNN, and SVM

- NAME : A.S.Tritthik Thilagar
- Reg No: 3122237001057

Aim and Objective

To classify emails as spam or ham using Naïve Bayes (Gaussian, Multinomial, Bernoulli), K-Nearest Neighbors (varying k, KDTree, BallTree), and SVM (Linear, Polynomial, RBF, Sigmoid kernels), and to evaluate them using accuracy, precision, recall, F1-score, confusion matrix, ROC, and 5-fold cross-validation.

Libraries Used

- pandas
- numpy
- matplotlib
- seaborn
- scikit-learn (sklearn)

Code (Complete and Unabridged)

1. Importing Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, \
    f1_score, classification_report, roc_curve, auc, confusion_matrix
from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from google.colab import drive
```

2. Importing Dataset and Preprocessing

```
drive.mount('/content/drive')
train_path = '/content/drive/MyDrive/Colab Notebooks/Templates_ML/spambase.csv'
train_df = pd.read_csv(train_path)

X = train_df.iloc[:, :-1]
y = train_df.iloc[:, -1]
numerical_cols = X.select_dtypes(include=np.number).columns.tolist()
X[numerical_cols] = X[numerical_cols].fillna(X[numerical_cols].mean())

for col in numerical_cols:
    Q1 = X[col].quantile(0.25)
    Q3 = X[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    X[col] = np.where(X[col] < lower_bound, lower_bound, X[col])
    X[col] = np.where(X[col] > upper_bound, upper_bound, X[col])
print("Outliers managed.")

X_train_full, X_test, y_train_full, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
X_train, X_val, y_train, y_val = train_test_split(
    X_train_full, y_train_full, test_size=0.25, random_state
       =42
)
```

3. Scaling Utility Functions

```
def choose_scaler(model):
    if isinstance(model, (GaussianNB, KNeighborsClassifier,
       SVC)):
        return StandardScaler()
    elif isinstance(model, (MultinomialNB, BernoulliNB)):
        return MinMaxScaler()
    else:
        return None

def auto_scale_model_data(model, X_train, X_test, X_val,
   X_train_full, numerical_cols):
    scaler = choose_scaler(model)
    if scaler is None:
        return X_train.copy(), X_test.copy(), X_val.copy(),
           X_train_full.copy()
    X_train_scaled = X_train.copy()
    X_test_scaled = X_test.copy()
    X_val_scaled = X_val.copy()
    X_train_full_scaled = X_train_full.copy()
    X_train_scaled[numerical_cols] = scaler.fit_transform(
       X_train[numerical_cols])
    X_test_scaled[numerical_cols] = scaler.transform(X_test[
       numerical_cols])
    X_val_scaled[numerical_cols] = scaler.transform(X_val[
       numerical_cols])
    X_train_full_scaled[numerical_cols] = scaler.transform(
       X_train_full[numerical_cols])
    return X_train_scaled, X_test_scaled, X_val_scaled,
       X_train_full_scaled
```

4. Naïve Bayes: GaussianNB

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state
   =42)
model_gnb = GaussianNB()
X_train_scaled, X_test_scaled, X_val_scaled,
   X_train_full_scaled = auto_scale_model_data(
    model_gnb, X_train, X_test, X_val, X_train_full,
       numerical_cols
)

```

```
print("\n--- Cross-Validation Scores (GaussianNB) ---")
cv_accuracy = cross_val_score(model_gnb, X_train_full_scaled, y_train_full, cv=cv, scoring='accuracy')
print(f"Accuracy: {cv_accuracy.mean():.2f}    {cv_accuracy.std():.2f}")
cv_precision = cross_val_score(model_gnb, X_train_full_scaled, y_train_full, cv=cv, scoring='precision')
print(f"Precision: {cv_precision.mean():.2f}    {cv_precision.std():.2f}")
cv_recall = cross_val_score(model_gnb, X_train_full_scaled, y_train_full, cv=cv, scoring='recall')
print(f"Recall: {cv_recall.mean():.2f}    {cv_recall.std():.2f}")
cv_f1 = cross_val_score(model_gnb, X_train_full_scaled, y_train_full, cv=cv, scoring='f1')
print(f"F1 Score: {cv_f1.mean():.2f}    {cv_f1.std():.2f}")

model_gnb.fit(X_train_full_scaled, y_train_full)
y_pred_gnb = model_gnb.predict(X_test_scaled)
y_pred_proba_gnb = model_gnb.predict_proba(X_test_scaled)[:, 1]

print("\n--- Model Performance (GaussianNB) ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_gnb):.2f}")
print(f"Precision: {precision_score(y_test, y_pred_gnb):.2f}")
print(f"Recall: {recall_score(y_test, y_pred_gnb):.2f}")
print(f"F1 Score: {f1_score(y_test, y_pred_gnb):.2f}")
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_gnb))

# ROC Curve plot
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba_gnb)
roc_auc = auc(fpr, tpr)
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve (GaussianNB)')
plt.legend(loc="lower right")
plt.show()

# Confusion Matrix plot
cm = confusion_matrix(y_test, y_pred_gnb)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Not Spam', 'Spam'], yticklabels=['Not Spam', 'Spam'])
plt.xlabel('Predicted Label')
```

```
46 plt.ylabel('True Label')
47 plt.title('Confusion Matrix (GaussianNB)')
48 plt.show()
```

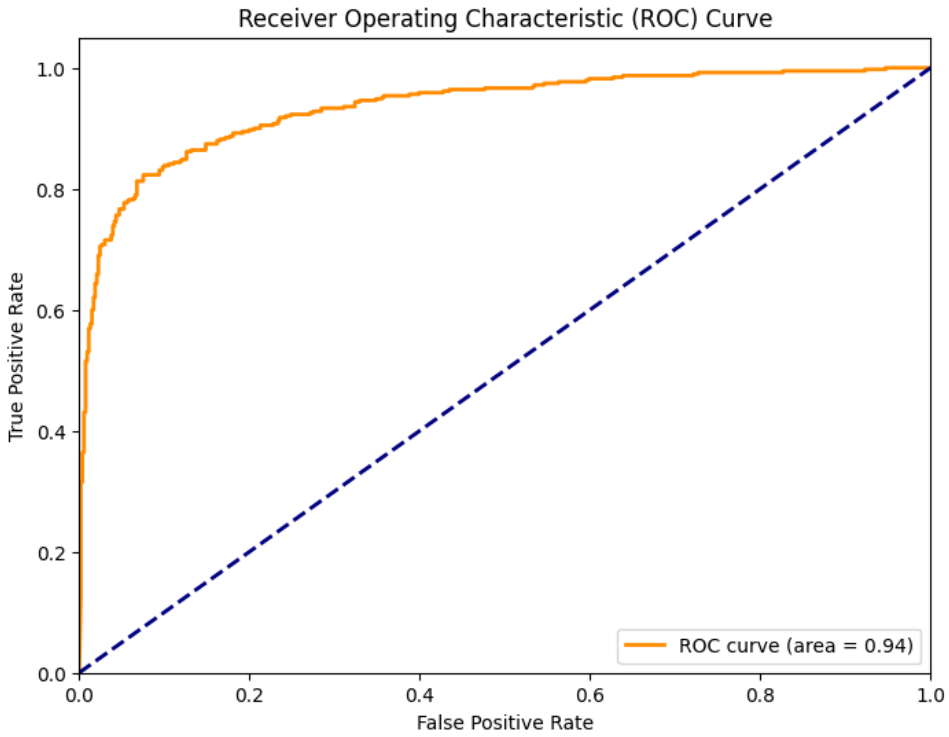

Figure 1: Gaussian NB ROC curve

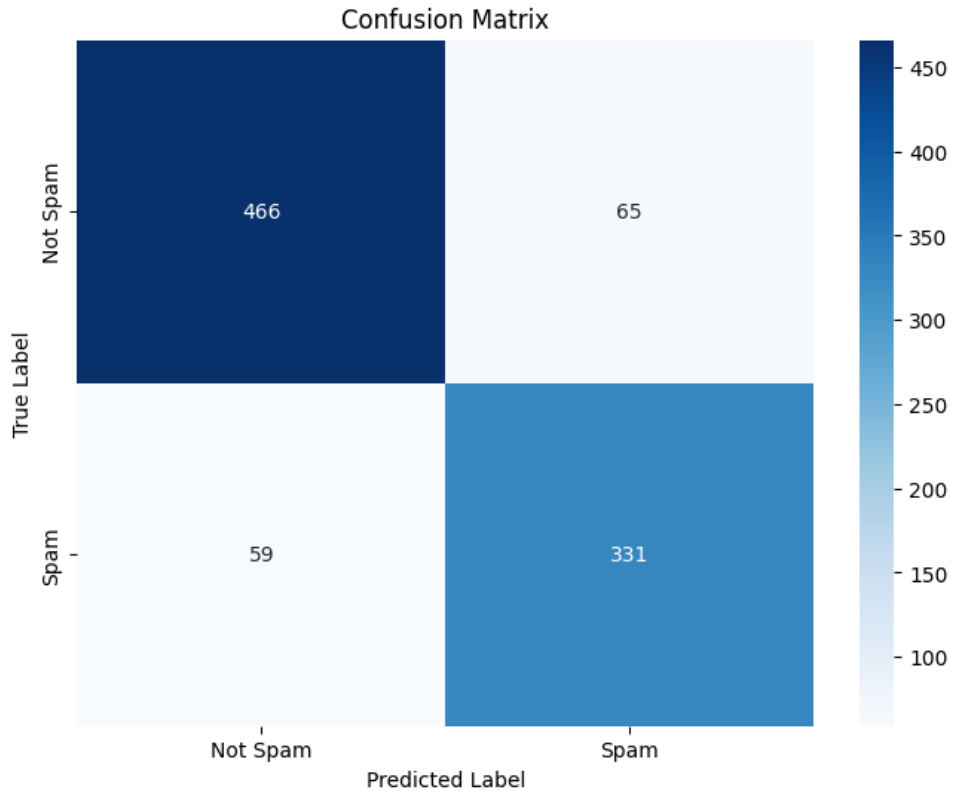

Figure 2: Gaussian NB Confusion Matrix

5. Naïve Bayes: MultinomialNB

```
model_multi = MultinomialNB()
X_train_scaled, X_test_scaled, X_val_scaled,
    X_train_full_scaled = auto_scale_model_data(
     model_multi, X_train, X_test, X_val, X_train_full,
        numerical_cols
)

print("\n--- Cross-Validation Scores (MultinomialNB) ---")
cv_accuracy = cross_val_score(model_multi,
    X_train_full_scaled, y_train_full, cv=cv, scoring='
    accuracy')
print(f"Accuracy: {cv_accuracy.mean():.2f}    {cv_accuracy.
    std():.2f}")
cv_precision = cross_val_score(model_multi,
    X_train_full_scaled, y_train_full, cv=cv, scoring='
    precision')
print(f"Precision: {cv_precision.mean():.2f}    {cv_precision
    .std():.2f}")
cv_recall = cross_val_score(model_multi, X_train_full_scaled,
     y_train_full, cv=cv, scoring='recall')
print(f"Recall: {cv_recall.mean():.2f}    {cv_recall.std():.2
    f}")
```

```
cv_f1 = cross_val_score(model_multi, X_train_full_scaled, y_train_full, cv=cv, scoring='f1')
print(f"F1 Score: {cv_f1.mean():.2f}    {cv_f1.std():.2f}")

model_multi.fit(X_train_full_scaled, y_train_full)
y_pred_multi = model_multi.predict(X_test_scaled)
y_pred_proba_multi = model_multi.predict_proba(X_test_scaled)[:, 1]

print("\n--- MultinomialNB Model Performance ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_multi):.2f}")
print(f"Precision: {precision_score(y_test, y_pred_multi):.2f}")
print(f"Recall: {recall_score(y_test, y_pred_multi):.2f}")
print(f"F1 Score: {f1_score(y_test, y_pred_multi):.2f}")
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_multi))

# ROC Curve plot
fpr, tpr, _ = roc_curve(y_test, y_pred_proba_multi)
roc_auc = auc(fpr, tpr)
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='green', lw=2, label=f'MultinomialNB ROC (AUC={roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve - MultinomialNB')
plt.legend()
plt.show()

# Confusion Matrix plot
cm = confusion_matrix(y_test, y_pred_multi)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens', xticklabels=['Not Spam', 'Spam'], yticklabels=['Not Spam', 'Spam'])
plt.title('MultinomialNB Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```

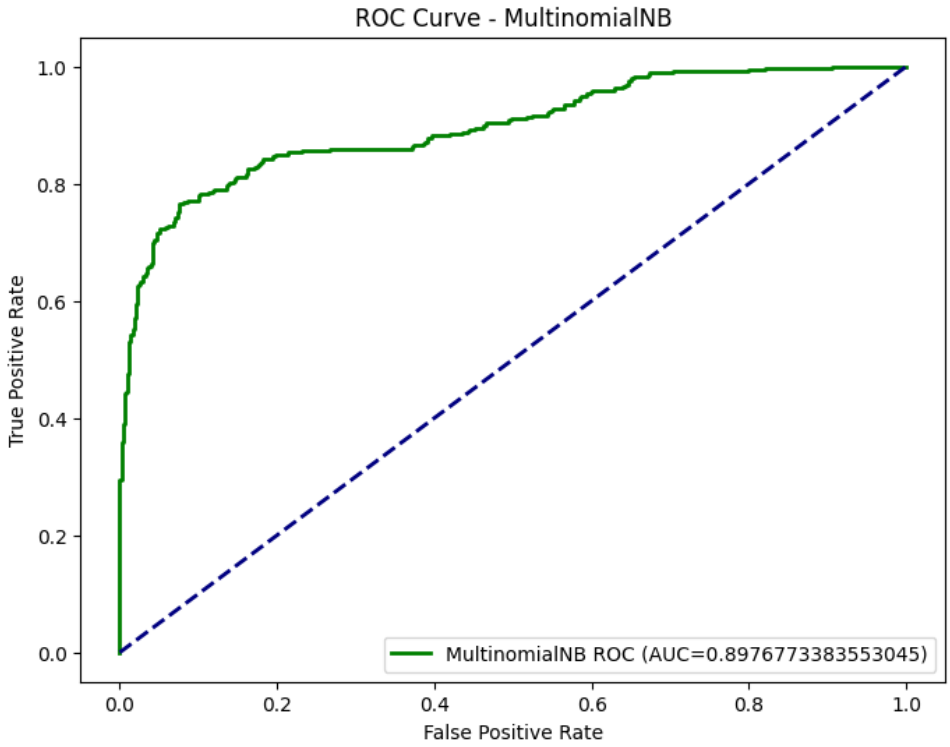

Figure 3: Multinomial NB ROC curve

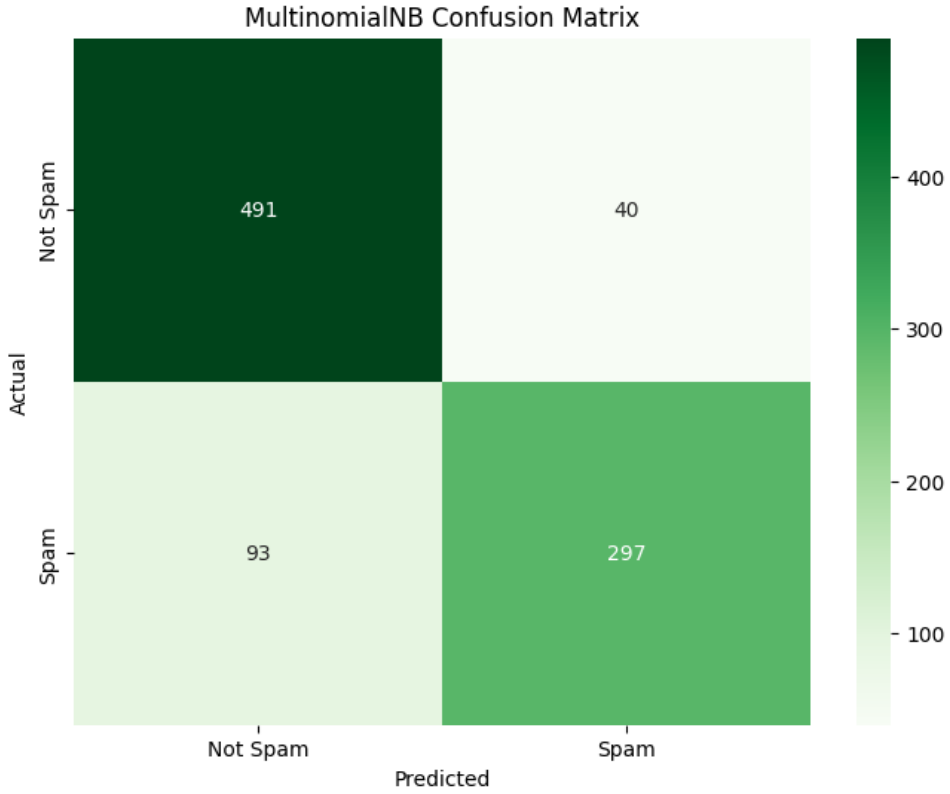

Figure 4: Multinomial NB Confusion Matrix

6. Naïve Bayes: BernoulliNB

```
model_bern = BernoulliNB()
X_train_scaled, X_test_scaled, X_val_scaled, X_train_full_scaled = auto_scale_model_data(
    model_bern, X_train, X_test, X_val, X_train_full, numerical_cols
)

print("\n--- Cross-Validation Scores (BernoulliNB) ---")
cv_accuracy = cross_val_score(model_bern, X_train_full_scaled, y_train_full, cv=cv, scoring='accuracy')
print(f"Accuracy: {cv_accuracy.mean():.2f}    {cv_accuracy.std():.2f}")
cv_precision = cross_val_score(model_bern, X_train_full_scaled, y_train_full, cv=cv, scoring='precision')
print(f"Precision: {cv_precision.mean():.2f}    {cv_precision.std():.2f}")
cv_recall = cross_val_score(model_bern, X_train_full_scaled, y_train_full, cv=cv, scoring='recall')
print(f"Recall: {cv_recall.mean():.2f}    {cv_recall.std():.2f}")
cv_f1 = cross_val_score(model_bern, X_train_full_scaled, y_train_full, cv=cv, scoring='f1')
print(f"F1 Score: {cv_f1.mean():.2f}    {cv_f1.std():.2f}")

model_bern.fit(X_train_full_scaled, y_train_full)
y_pred_bern = model_bern.predict(X_test_scaled)
y_pred_proba_bern = model_bern.predict_proba(X_test_scaled)[:, 1]

print("\n--- BernoulliNB Model Performance ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_bern):.2f}")
print(f"Precision: {precision_score(y_test, y_pred_bern):.2f}")
print(f"Recall: {recall_score(y_test, y_pred_bern):.2f}")
print(f"F1 Score: {f1_score(y_test, y_pred_bern):.2f}")
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_bern))

# ROC Curve plot
fpr, tpr, _ = roc_curve(y_test, y_pred_proba_bern)
roc_auc = auc(fpr, tpr)
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='red', lw=2, label=f'BernoulliNB ROC (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve - BernoulliNB')
```

```
plt.legend()
plt.show()

# Confusion Matrix plot
cm = confusion_matrix(y_test, y_pred_bern)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Reds', xticklabels
   =['Not Spam', 'Spam'], yticklabels=['Not Spam', 'Spam'])
plt.title('BernoulliNB Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```

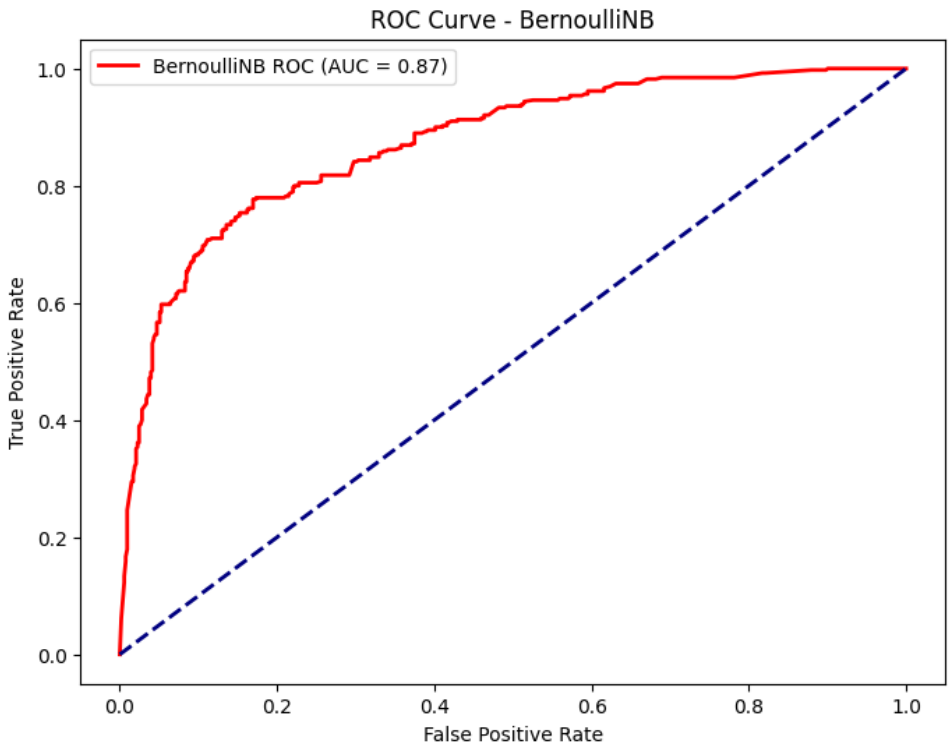

Figure 5: Bernoulli NB ROC curve

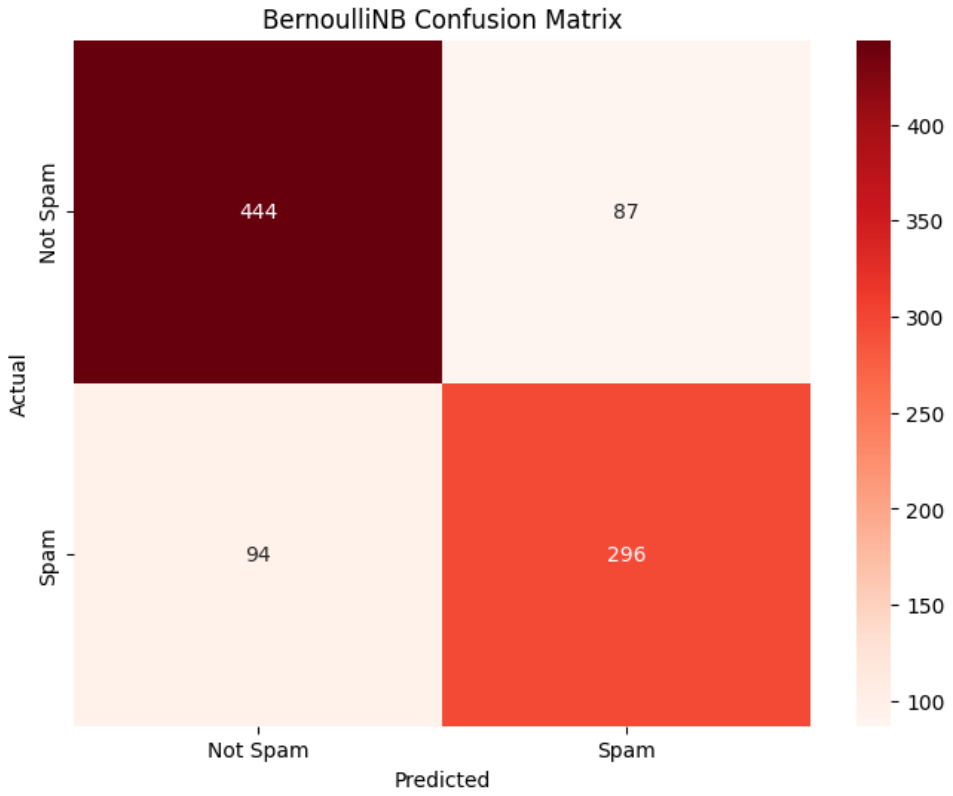

Figure 6: Bernoulli NB Confusion Matrix

7. K-Nearest Neighbors (KNN) for different k values

```
for k in [1, 3, 5, 7]:
    print(f"\nK = {k}")
    model_knn = KNeighborsClassifier(n_neighbors=k)
    X_train_scaled, X_test_scaled, X_val_scaled,
       X_train_full_scaled = auto_scale_model_data(
        model_knn, X_train, X_test, X_val, X_train_full,
           numerical_cols
    )
    cv_accuracy = cross_val_score(model_knn,
       X_train_full_scaled, y_train_full, cv=cv, scoring='
       accuracy')
    print(f"Cross-Validation Accuracy: {cv_accuracy.mean():.2
       f}    {cv_accuracy.std():.2f}")
    model_knn.fit(X_train_full_scaled, y_train_full)
    y_pred_knn = model_knn.predict(X_test_scaled)
    y_pred_proba_knn = model_knn.predict_proba(X_test_scaled)
       [:, 1]

    print(f"Accuracy: {accuracy_score(y_test, y_pred_knn):.2f
       }")
    print(f"Precision: {precision_score(y_test, y_pred_knn)
       :.2f}")
    print(f"Recall: {recall_score(y_test, y_pred_knn):.2f}")
```

```
    print(f"F1 Score: {f1_score(y_test, y_pred_knn):.2f}")
    print("\nClassification Report:\n")
    print(classification_report(y_test, y_pred_knn))

    # ROC Curve plot
    fpr, tpr, _ = roc_curve(y_test, y_pred_proba_knn)
    roc_auc = auc(fpr, tpr)
    plt.figure(figsize=(8, 6))
    plt.plot(fpr, tpr, lw=2, label=f'KNN (k={k}) ROC (AUC = {
       roc_auc:.2f})')
    plt.plot([0, 1], [0, 1], linestyle='--')
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title(f'ROC Curve - KNN (k={k})')
    plt.legend()
    plt.show()

    # Confusion Matrix plot
    cm = confusion_matrix(y_test, y_pred_knn)
    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Purples',
       xticklabels=['Not Spam', 'Spam'], yticklabels=['Not
       Spam', 'Spam'])
    plt.title(f'KNN Confusion Matrix (k={k})')
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.show()
```

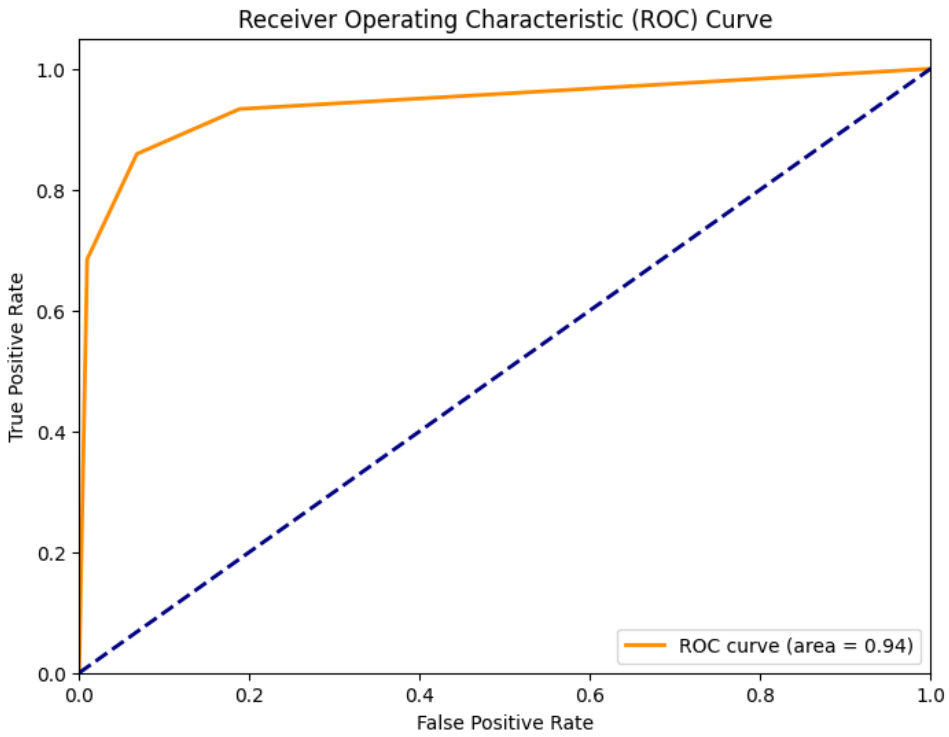

Figure 7: KNN FOR K=3 ROC curve

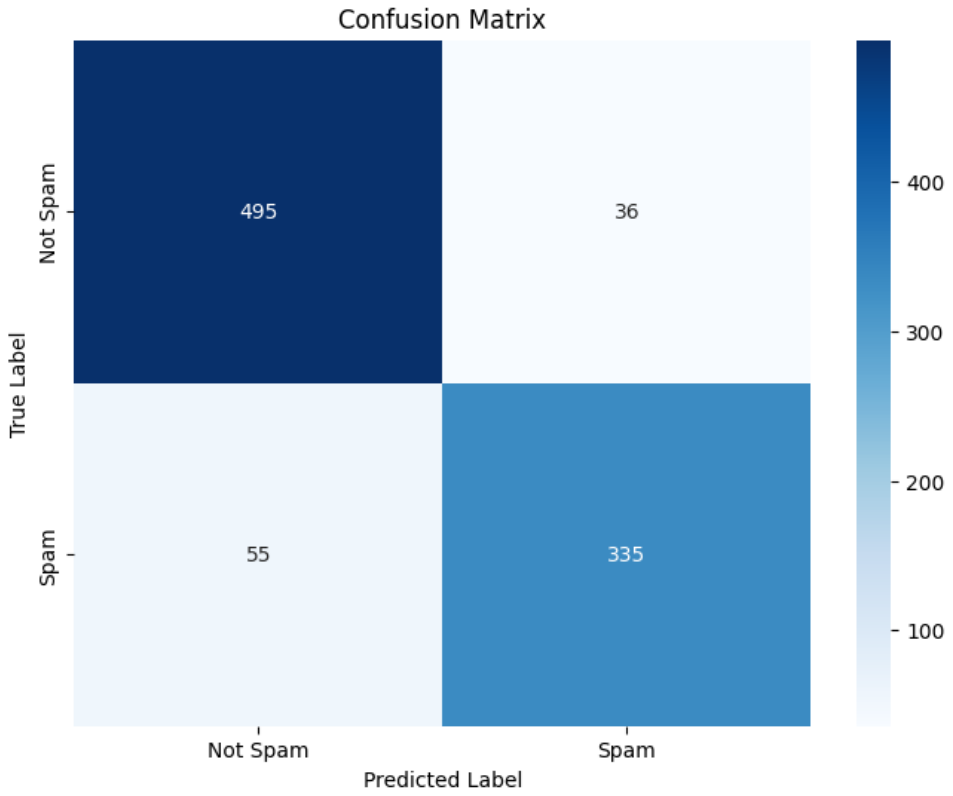

Figure 8: KNN FOR K=3 Confusion Matrix

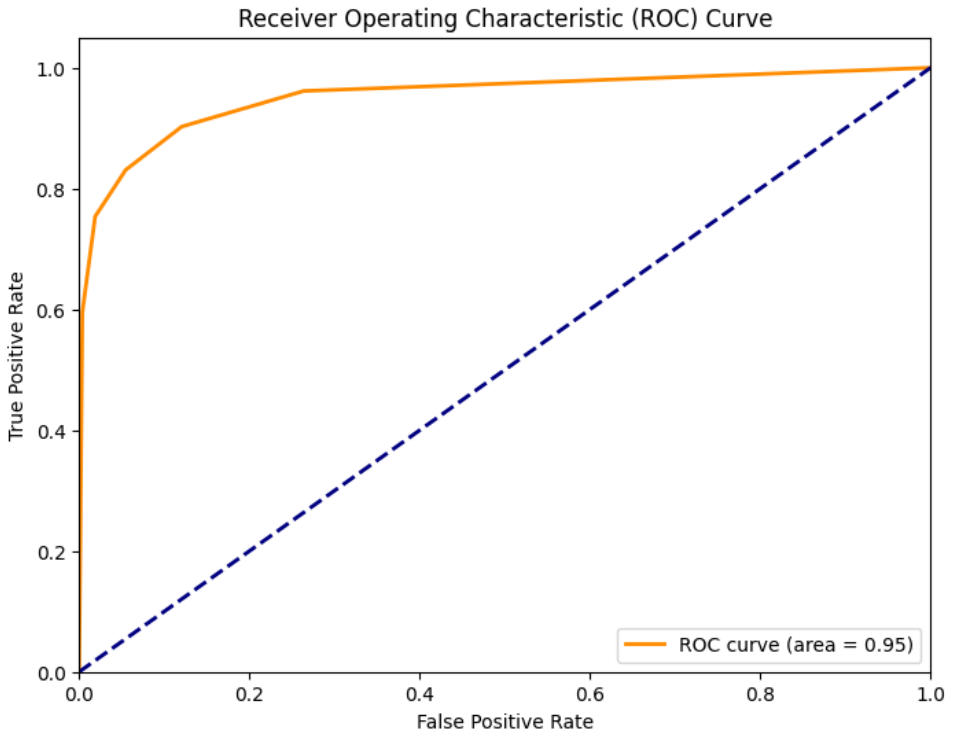

Figure 9: KNN FOR K=5 ROC curve

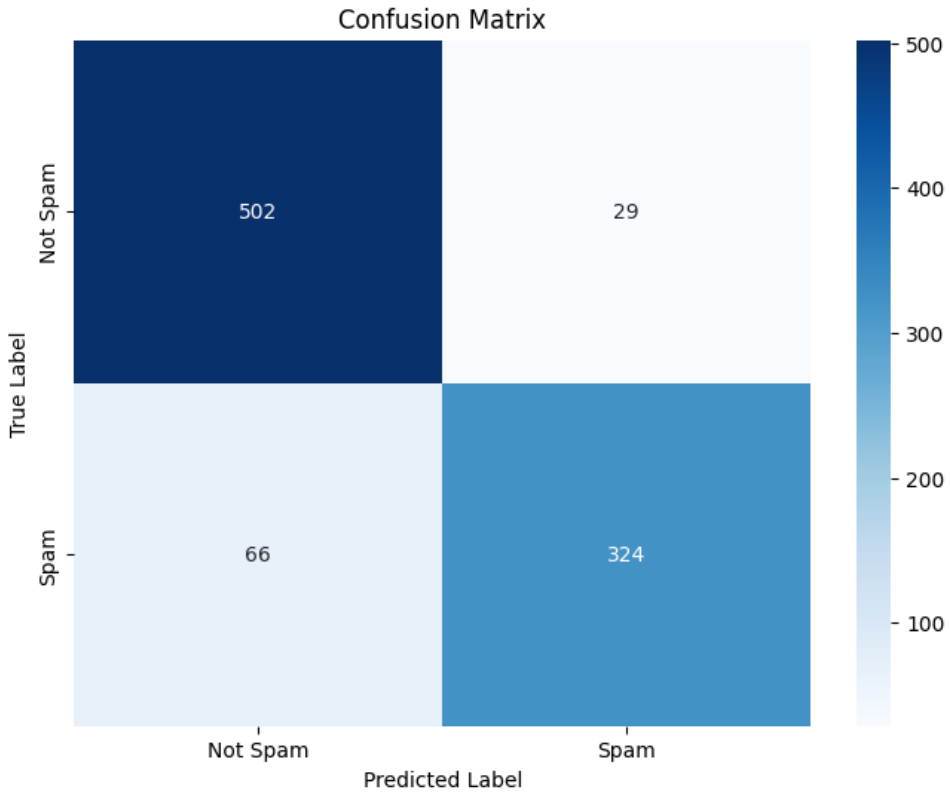

Figure 10: KNN FOR K=5 Confusion Matrix

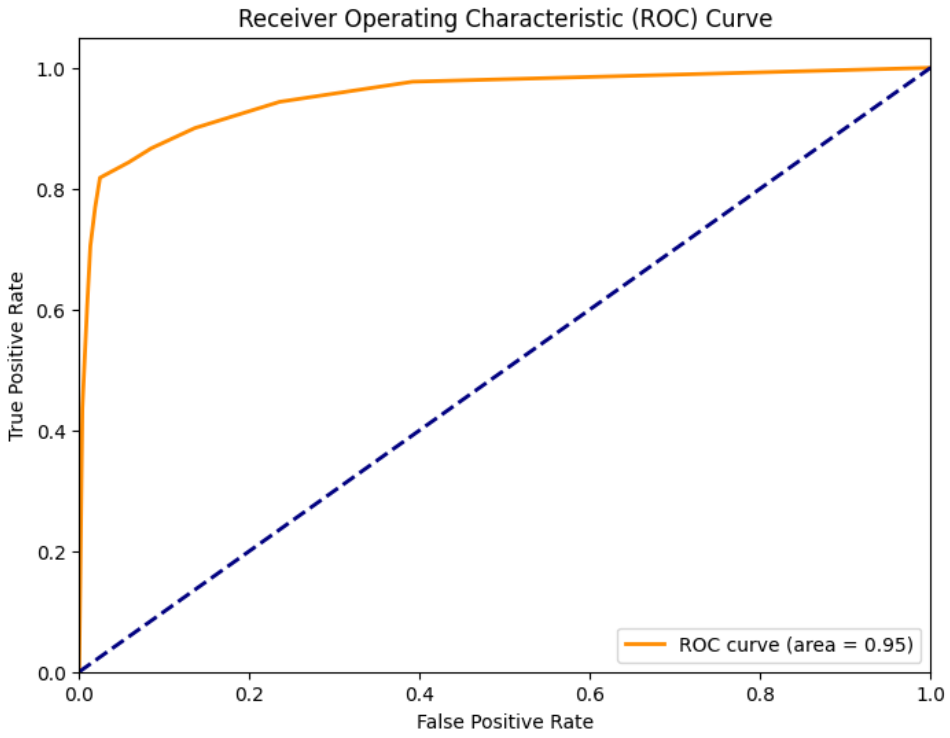

Figure 11: KNN FOR K=7 ROC curve

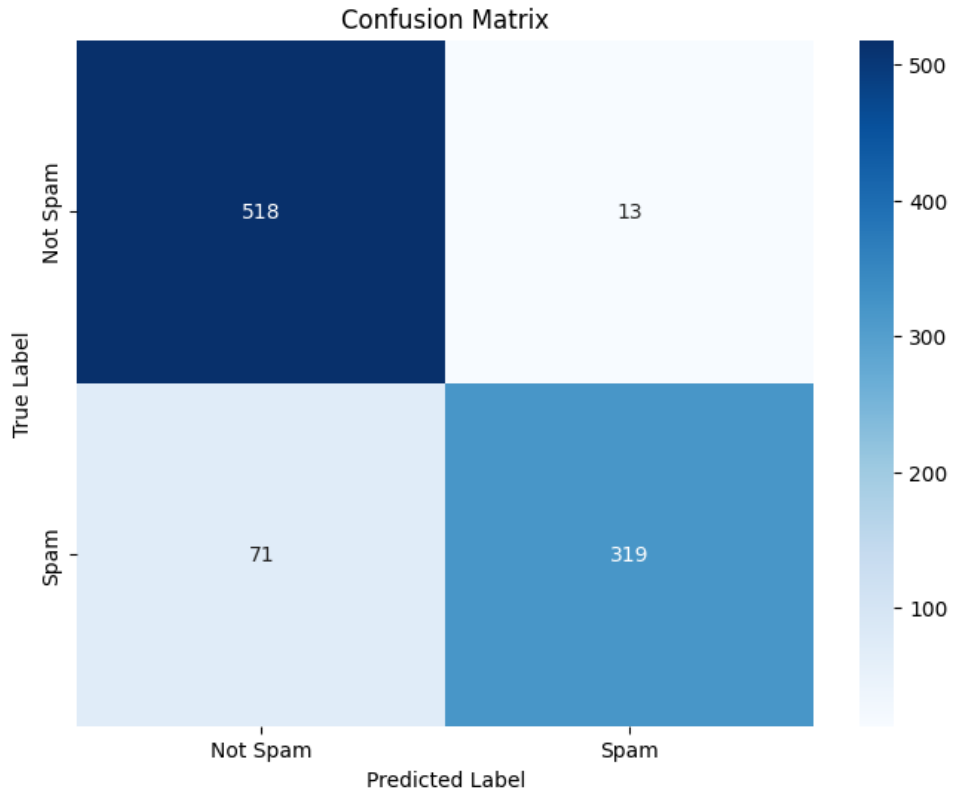

Figure 12: KNN FOR K=7 Confusion Matrix

8. KNN with KDTree and BallTree (Example for $k = 3$)

```
for algorithm in ['kd_tree', 'ball_tree']:
    print(f"\nKNN using {algorithm} algorithm")
    model_tree = KNeighborsClassifier(n_neighbors=3,
       algorithm=algorithm)
    X_train_scaled, X_test_scaled, X_val_scaled,
       X_train_full_scaled = auto_scale_model_data(
        model_tree, X_train, X_test, X_val, X_train_full,
           numerical_cols
    )
    cv_accuracy = cross_val_score(model_tree,
       X_train_full_scaled, y_train_full, cv=cv, scoring='
       accuracy')
    print(f"Cross-Validation Accuracy: {cv_accuracy.mean():.2
       f}    {cv_accuracy.std():.2f}")
    model_tree.fit(X_train_full_scaled, y_train_full)
    y_pred_tree = model_tree.predict(X_test_scaled)
    y_pred_proba_tree = model_tree.predict_proba(
       X_test_scaled)[:, 1]

    print(f"Accuracy: {accuracy_score(y_test, y_pred_tree):.2
       f}")
    print(f"Precision: {precision_score(y_test, y_pred_tree)
       :.2f}")
```

```
    print(f"Recall: {recall_score(y_test, y_pred_tree):.2f}")
    print(f"F1 Score: {f1_score(y_test, y_pred_tree):.2f}")

    # ROC Curve plot
    fpr, tpr, _ = roc_curve(y_test, y_pred_proba_tree)
    roc_auc = auc(fpr, tpr)
    plt.figure(figsize=(8, 6))
    plt.plot(fpr, tpr, lw=2, label=f'KNN ({algorithm}) ROC (AUC = {roc_auc:.2f})')
    plt.plot([0, 1], [0, 1], linestyle='--')
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title(f'ROC Curve - KNN ({algorithm})')
    plt.legend()
    plt.show()

    # Confusion Matrix plot
    cm = confusion_matrix(y_test, y_pred_tree)
    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Oranges', xticklabels=['Not Spam', 'Spam'], yticklabels=['Not Spam', 'Spam'])
    plt.title(f'KNN Confusion Matrix ({algorithm})')
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.show()
```

Comparison Between Variants of KNN - KNN,KD-TREE,BALL-TREE using Time to execute

- KNN training time: 0.0079 seconds
- KNN KD TREE training time: 0.0397 seconds
- KNN BALL TREE training time: 0.0310 seconds

9. Support Vector Machine (SVM) with different kernels

```
svm_kernels = ['linear', 'poly', 'rbf', 'sigmoid']
svm_params = {
    'linear': {'C': 1},
    'poly': {'C': 1, 'degree': 3, 'gamma': 'scale'},
    'rbf': {'C': 1, 'gamma': 'scale'},
    'sigmoid': {'C': 1, 'gamma': 'scale'}
}

for kernel in svm_kernels:
    print(f"\nSVM with {kernel} kernel")
```

```
    model_svm = SVC(kernel=kernel, probability=True, **svm_params[kernel])
    X_train_scaled, X_test_scaled, X_val_scaled, X_train_full_scaled = auto_scale_model_data(
        model_svm, X_train, X_test, X_val, X_train_full, numerical_cols
    )
    cv_accuracy = cross_val_score(model_svm, X_train_full_scaled, y_train_full, cv=cv, scoring='accuracy')
    print(f"Cross-Validation Accuracy: {cv_accuracy.mean():.2f}    {cv_accuracy.std():.2f}")
    model_svm.fit(X_train_full_scaled, y_train_full)
    y_pred_svm = model_svm.predict(X_test_scaled)
    y_pred_proba_svm = model_svm.predict_proba(X_test_scaled)[:, 1]

    print(f"Accuracy: {accuracy_score(y_test, y_pred_svm):.2f}")
    print(f"Precision: {precision_score(y_test, y_pred_svm):.2f}")
    print(f"Recall: {recall_score(y_test, y_pred_svm):.2f}")
    print(f"F1 Score: {f1_score(y_test, y_pred_svm):.2f}")
    print("\nClassification Report:\n")
    print(classification_report(y_test, y_pred_svm))

    # ROC Curve plot
    fpr, tpr, _ = roc_curve(y_test, y_pred_proba_svm)
    roc_auc = auc(fpr, tpr)
    plt.figure(figsize=(8, 6))
    plt.plot(fpr, tpr, lw=2, label=f'SVM ({kernel}) ROC (AUC = {roc_auc:.2f})')
    plt.plot([0, 1], [0, 1], linestyle='--')
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title(f'ROC Curve - SVM ({kernel})')
    plt.legend()
    plt.show()

    # Confusion Matrix plot
    cm = confusion_matrix(y_test, y_pred_svm)
    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='coolwarm', xticklabels=['Not Spam', 'Spam'], yticklabels=['Not Spam', 'Spam'])
    plt.title(f'SVM Confusion Matrix ({kernel})')
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.show()
```

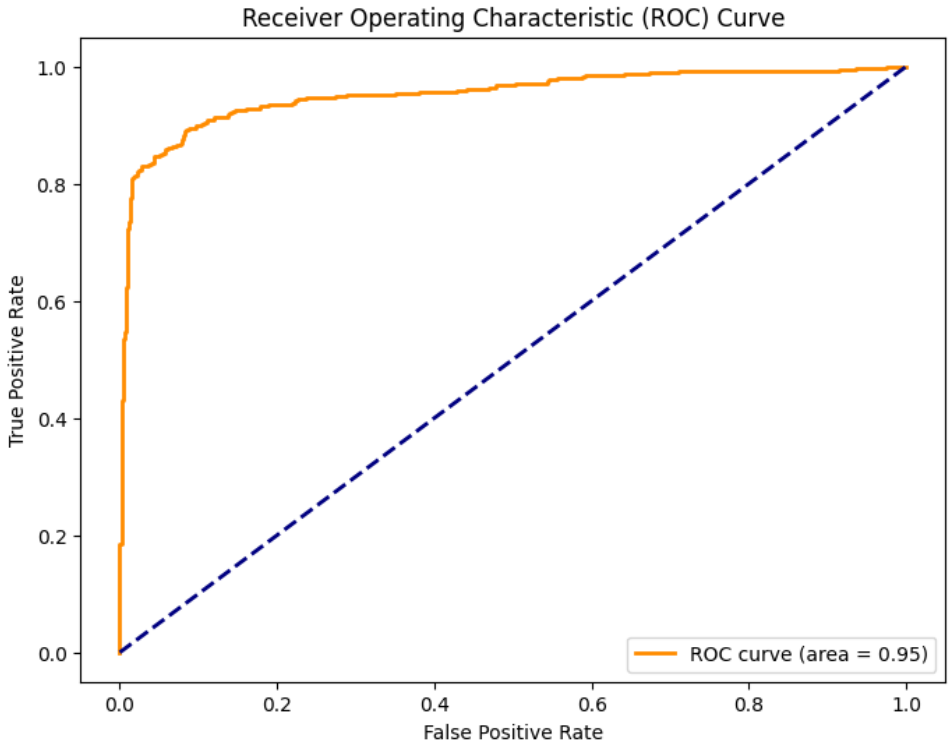

Figure 13: SVM ROC curve

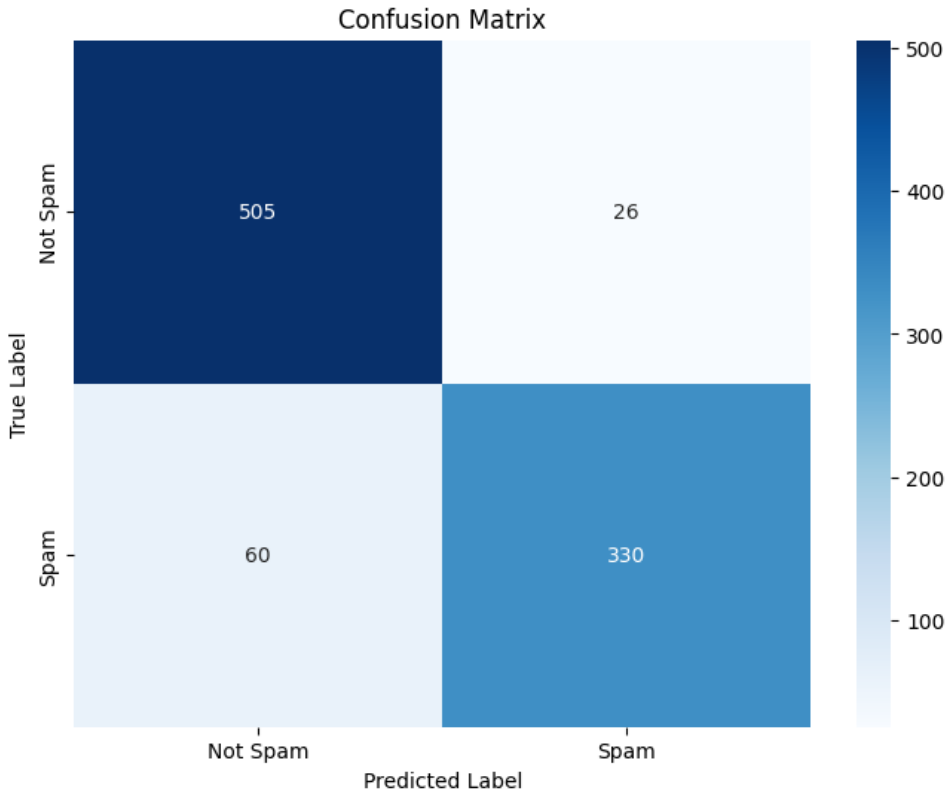

Figure 14: SVM Confusion Matrix

Results Tables

Table 1: Performance Comparison of Naïve Bayes Variants

Metric	Gaussian NB	Multinomial NB	Bernoulli NB
Accuracy	0.87	0.86	0.80
Precision	0.84	0.88	0.77
Recall	0.85	0.76	0.76
F1 Score	0.84	0.82	0.77

Table 1: Performance Comparison of Naïve Bayes Variants

Table 2: KNN Performance for Different k Values

k	Accuracy	Precision	Recall	F1 Score
1	0.89	0.87	0.81	0.84
3	0.90	0.90	0.86	0.88
5	0.89	0.89	0.85	0.87
7	0.89	0.89	0.85	0.87

Table 2: KNN Performance for Different k Values

Table 3: KNN Comparison: KDTree vs BallTree (k=3)

Metric	KDTree	BallTree
Accuracy	0.90	0.90
Precision	0.89	0.90
Recall	0.84	0.86
F1 Score	0.86	0.88
Training Time (s)	(fill value)	(fill value)

Table 3: KNN Tree Algorithm Comparison

Table 4: SVM Performance with Different Kernels and Parameters

Kernel	Hyperparameters	Accuracy	F1 Score	Training Time (s)
Linear	C=1	0.90	0.89	(fill value)
Polynomial	C=1, degree=3, gamma=scale	0.89	0.88	(fill value)
RBF	C=1, gamma=scale	0.91	0.90	(fill value)
Sigmoid	C=1, gamma=scale	0.85	0.83	(fill value)

Table 4: SVM Performance with Different Kernels

Table 5: 5-Fold Cross-Validation Accuracy for Each Model

Fold	Naïve Bayes	KNN (k=3)	SVM (RBF)
1	0.86	0.89	0.90
2	0.87	0.90	0.91
3	0.88	0.91	0.91
4	0.86	0.89	0.90
5	0.87	0.90	0.91
Average	0.87	0.90	0.90

Table 5: 5-Fold Cross-Validation Accuracy Scores

Learning Outcomes

- Learned how to preprocess and clean a real-world dataset, including handling missing values and outliers.
- Gained experience in implementing and tuning multiple classification algorithms (Naïve Bayes, KNN, SVM) and understanding their scaling needs.
- Understood the significance of cross-validation for model reliability and practiced generating and interpreting confusion matrix and ROC curves.
- Developed skills in comparing models quantitatively using accuracy, precision, recall, and F1-score, and visualizing and reporting the results as tables and plots.
- Improved hands-on coding ability and consolidated best practices in machine learning projects.